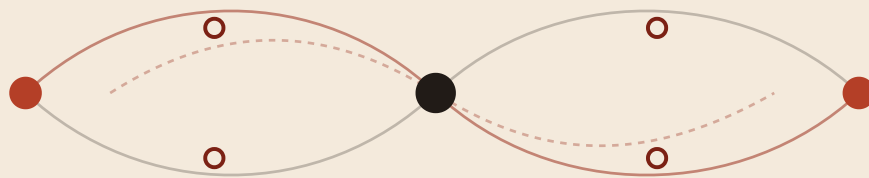


A High-Level View of Work Context Infrastructure



1. From perception to intention

All work begins, at least in some form, with an idea about something that could be different. Someone observes something in their life or in the reality around them and perceives a possibility, a need, a problem, or a condition worth preserving. From that perception, an objective begins to take shape.

The objective does not appear in isolation. Perception is situated. People encounter reality through lenses influenced by their experience, knowledge, biological constitution, social position, and cultural context. These factors do not mechanically determine one conclusion, but they influence what becomes visible, which distinctions seem important, and which possibilities can be imagined.

The system does not need to model human perception itself. It needs to preserve the attributable transition from a situated perception to a declared and revisable objective.

At first, the objective often appears as a single monolithic intention: one large, still formless whole with little internal detail or clarity. A person may sense a direction before they can describe the destination precisely. They may know that something should change without yet knowing the work, constraints, or sequence required to change it. The intention carries a purpose, but its parts, relationships, and much of its meaning remain implicit.

2. From a monolithic intention to a composition of intentions

As time and effort are invested, the monolithic intention begins to acquire contour. Distinctions appear inside it: outcomes, constraints, questions, and possible routes that were previously fused. Some of these distinctions become separate intentions, each with its own objective, scope, and

possible work. Together, they compose the initial objective rather than merely sitting beneath it. The relationship is also recursive: what is learned while pursuing one of these intentions may clarify, challenge, or change the objective from which it emerged.

A more specific intention may become a possible route. When that route needs its own proposal identity, authority search, and lifecycle, it may become a Plan. It may instead remain a section, workstream, task, or another bounded form of work. In either case, the route can be developed into steps, each described more closely in terms of what needs to be done and, sometimes, how it might be done.

```
perception
-> objective
-> monolithic intention
-> differentiated intentions
<-> composed and revisable objective
-> possible route
-> Plan, workstream, or another decomposable form
-> steps
-> detailed work
-> tasks
-> attempts
-> effects and artifacts
```

A possible route may instead become one directly bounded task; the sequence above is an illustrative decomposition, not a required lifecycle.

The intention does not merely become longer. It becomes composed of more parts. What was once held together as one idea begins to appear across conversations, Plans, research, decisions, specifications, assignments, tasks, attempts, and changed files.

This makes focused work possible, but it changes what is visible. Attention moves closer to the individual parts. A task may remain understandable by itself while the objective that made it worth doing becomes difficult to see. Several tasks may be completed correctly even after the assumption, feature, or larger purpose they were meant to serve has changed.

The work becomes more precise locally while its purpose becomes less visible globally.

3. The problem the system must solve

The central purpose of the proposed system is to preserve the relationship between local work and the larger contexts that explain it.

A local piece of work should remain connected to:

- the objective from which it ultimately emerged;
- the interpretations, assumptions, and evidence that shaped that objective;

- the Plans, discoveries, specifications, and decisions that refined it;
- the authority that permitted a person, agent, or tool to act;
- the attempts, effects, and artifacts through which the work became concrete; and
- the evidence used to judge whether the result advanced anything larger.

This is not only a task-management problem. A task manager can record assignments while losing their justification. It is not only a document-management problem. Documents can preserve text while leaving their semantic and operational relationships implicit. It is not only a workflow or multi-agent problem. A workflow can execute the wrong route correctly.

The deeper problem is the loss of context between levels of work.

Every piece of work should remain connected to the larger contexts that give it purpose, meaning, and legitimate authority.

The system must preserve five independently inspectable paths: **purpose**, **authority**, **assignment**, **causation**, and **realization**. They may converge on the same outcome, but none can be inferred from another. An execution can, for example, be authorized without having been assigned to the purpose-bearing task. The remainder of this essay explains the properties, responsibilities, and validation strategy required to preserve these paths.

When inspected, a path may be witnessed, missing, conflicted, or superseded. These are explanatory distinctions rather than a fixed response vocabulary.

Reading map

- Sections 1–16 establish the problem and the properties the system may need.
- Sections 17–30 describe the candidate shape and its responsibilities.
- Sections 31–34 name preservation and formalization questions.
- Sections 35–36 define the first discriminating test.
- Sections 37–38 preserve open questions, ownership gaps, and limits.

Alternative framings considered

Framing	Why it is insufficient on its own
Task-management system	Tracks assignments and status but may not preserve purpose, evidence, or authority.
Document repository	Preserves artifacts but may leave their relationships and operational consequences implicit.
Workflow engine	Coordinates execution after meaning and legitimacy have already been assumed.
Multi-agent orchestrator	Explains one important execution mechanism, not the full path from intention to effect.

FIVE INDEPENDENTLY INSPECTABLE PATHS

01

Purpose

why the work exists

02

Authority

what permits action

03

Assignment

who carries
responsibility

04

Causation

what produced
change

05

Realization

what supports the
outcome

CANDIDATE OUTCOME

4. Moving from the part to the whole

From any task, attempt, artifact, or effect, a person should be able to move upward through its context:

```
Why does this exist?  
Which work asked for it?  
Which Plan or route produced that work?  
Which objective does the route serve?  
Which decision authorized this action?  
Which assumptions and evidence shaped those choices?
```

This upward path checks purpose and authority. It makes orphan work visible. A task with no defensible upward path may still be technically clear, but the system cannot explain why it should be performed.

The path need not be one fixed chain. A task may acquire its purpose through one set of relations and its authority through another. What matters is that the relevant paths remain inspectable and that the system does not invent missing links.

5. Moving from the whole to its parts

From an objective, a person should also be able to move downward:

```
Which Plans attempt to realize this objective?  
Which work is active, blocked, abandoned, or complete?  
Which decisions changed the route?  
Which tasks and attempts are operating now?  
Which effects and artifacts were produced?  
What evidence indicates that the objective is being advanced?
```

This downward path checks realization. An objective with no path toward concrete work may be an aspiration with nothing attempting to implement it. A Plan with no tasks may be incomplete. A specification with no implementation and an implementation with no verification are different, inspectable gaps.

Upward and downward traversal are complementary. The upward path asks whether local work remains justified. The downward path asks whether high-level intent has become real.

6. More than one context

Work does not belong to one universal hierarchy.

A task can simultaneously:

- belong to a sprint;
- implement a specification;
- change a feature;
- answer a discovery;
- consume a research result;
- satisfy part of a Plan;
- be executed by an agent; and
- be authorized by a user decision.

These connections mean different things. They should not be reduced to one `parent` relation, and they should not be inferred from one folder path. A file has one physical location, but the work it represents can participate in several contexts at the same time.

The system must therefore support multiple simultaneous organizations of the same underlying work. A project may be viewed by feature, Plan, objective, sprint, agent, artifact type, authority scope, or unresolved question without creating a different identity for each view.

7. Context as composition

At this explanatory altitude, it is not enough to treat context only as a container around a piece of work. The future ontology must determine whether and how context is constituted by the relevant relationships through which that work participates in a larger system.

A task is not fully explained by its internal description. Its meaning also depends on:

- what produced it;

- what it contributes to;
- what constrains it;
- what it depends on;
- what authorized it;
- what evidence it uses; and
- what later work depends on it.

No single relation necessarily provides enough context. The explanation may depend on a path across several relations, each with a distinct meaning. The system must preserve the path without pretending that every path licenses the same conclusion.

Alternative framings considered

Framing	Why it is insufficient on its own
One organizational tree	A piece of work may participate in several non-hierarchical contexts.
Folder structure as context	Physical placement is one projection and cannot safely encode every semantic relation.
Put all context in the task description	Repeats and freezes information that belongs to independently changing objects.
Infer relationships from proximity	Nearby files or events are not necessarily semantically or authoritatively connected.

8. Stable identity under changing descriptions

The system must distinguish a thing from its current description.

Names, paths, classifications, and relationships can change while the thing remains addressable. A feature can move to another project. A Plan can be renamed. An early artifact can be classified as Research only after it has existed for some time. The same object can appear in several views without becoming several objects.

Stable identity makes it possible to preserve history through these changes. It does not require every object to receive a permanent universal identifier immediately. It requires the system to make identity decisions explicit and to avoid treating mutable descriptions as if they were the identity itself.

9. Relationships with distinct meanings

Once multiple contexts are allowed, relationships need semantics.

Connections such as these are not interchangeable:

```
part-of  
implements  
uses  
answers  
depends-on  
authorized-by  
produced-by  
supports  
contradicts  
supersedes
```

`Produced-by` does not imply `accepted-as`. `Part-of` does not imply `authorized-by`. `Uses` does not imply `supports`. A system that collapses these distinctions may create convenient paths while manufacturing conclusions that no accepted relation warrants.

The exact meaning of relation and the relation vocabulary remain deferred. The property required at this altitude is that relations preserve enough meaning to constrain traversal, derivation, authority, and change.

10. Progressive definition

The system cannot require complete knowledge at the moment something is first noticed.

An idea may deserve preservation before anyone knows whether it is Research, Discovery, a Plan, a requirement, a task, a hypothesis, or another kind of object. Requiring a complete schema at creation makes early capture expensive and encourages important context to remain in private memory or transient conversation.

A possible progression is:

```
partially understood object  
-> observations and candidate descriptions  
-> stronger classifications and relations  
-> reviewed or accepted contracts  
-> operational use under explicit authority
```

This is not one mandatory maturity ladder. It expresses increasing commitment. Cheap descriptions should remain distinguishable from structural conformance, accepted truth, and permission to act.

11. Provenance and responsibility

Important descriptions, relationships, and decisions should retain their origin.

The system should be able to distinguish:

```
a relation is asserted
a relation is supported by evidence
a relation is accepted in one context
a relation is authorized for operational use
```

It should preserve who or what made the assertion, when it was made, which evidence was available, and which process accepted or rejected it. An agent-generated statement remains attributable to the agent. A validator result remains evidence from that validator. A user decision remains a decision by that user.

Provenance does not by itself make a claim true or an effect authorized. It makes the basis for later judgments inspectable.

12. History rather than silent replacement

Context cannot be preserved by freezing it. It must be preserved by recording how it changes.

```
Feature-F was part of Project-A
Feature-F is now part of Project-B
Decision-D1 authorized the original route
Decision-D2 replaced that route
Task-T began under D1
Task-U began under D2
```

The current representation can be derived without erasing the earlier one. This makes it possible to distinguish work that violated the current route from work that was valid under an earlier route, and to understand why later decisions replaced earlier ones.

History should preserve accepted change, not every untrusted observation as if it had equal authority. The source, acceptance boundary, and epistemic status of a historical fact remain part of its meaning.

Alternative framings considered

Framing	Why it is insufficient on its own
Mutable current-state records only	They erase why earlier work existed and how the current representation emerged.
Append every observation as truth	An event was recorded does not mean its interpretation was correct or accepted.
Require settled types before identity	Early work remains outside the system precisely when context is most likely to be lost.

Framing	Why it is insufficient on its own
Treat provenance as validity	Knowing who asserted something does not establish that the assertion is true or authorized.

13. Decisions as part of context

Approvals, rejections, corrections, and selections should not disappear inside conversations.

A decision should remain connected to what was decided, who decided it, the context in which it applied, and the later work that relied on it. A rejection is also durable information. It can explain why work did not happen and reveal a boundary that should constrain later proposals.

The system should not collapse a decision into the state of its target. “Accepted” is not only a label on a Plan; it is also an attributable occurrence with a scope, basis, and time.

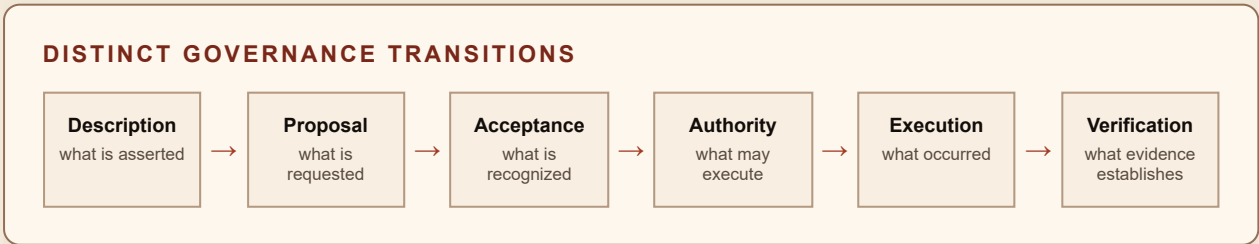
14. Acceptance is not authority to execute

Several states that look similar in ordinary conversation must remain separate:

```
described
!= proposed
!= accepted
!= authorized to execute
!= executed
!= verified
```

A Plan may be accepted as the best current route without authorizing any Dispatch. A research result may be accepted as evidence without authorizing a file change. A detailed task may still lack the authority, capability, or resources required to run.

The system should make these boundaries visible because later interfaces cannot recover distinctions that the underlying records never preserved.



15. Authority across levels

A user may authorize one Dispatch. The user may also authorize a bounded workflow:

Continue this research, review, and implementation workflow without asking before every task.

These decisions operate at different levels.

```
User Decision
-> Authorized Workflow
-> Dispatch
-> Attempt
-> Effect
```

The broader decision is not unlimited. It applies to the objective, scope, kinds of work, limits, conditions, and duration the user accepted. Later Dispatches should remain connected to it, and the system should be able to explain why each local effect is still covered.

16. Authority as a containment question

For local work to proceed under broader authority, the system must determine whether the work remains inside the approved boundary.

Relevant dimensions may include:

- objective and intended outcome;
- repositories and path scopes;
- kinds of work and effects;
- tools and capabilities;
- network and mutation boundaries;
- agent, time, token, and round limits;
- prohibited actions;
- expiration and stop conditions.

An omitted or ambiguous dimension should not silently become permission. How containment is represented and checked remains a load-bearing design choice: `OD-01`.

Alternative framings considered

Framing	Why it is insufficient on its own
Approval as a boolean	Loses scope, conditions, duration, and the object of the decision.
Plan acceptance implies execution	Confuses agreement about a route with permission to produce effects.
Authority automatically inherits through ancestry	A parent relation does not necessarily delegate tools, budget, or effect permission.
Ask before every action	Preserves control but can make bounded, repeated work unnecessarily expensive.

17. A recursive grammar of work

A familiar progression offers a useful first reading:

```
Intent
-> Plan
-> Research
-> Discovery / Design
-> Specification
-> Implementation
-> Verification
```

It is not one irreversible pipeline. Research may challenge a Plan. Verification may reopen a Specification. A local implementation question may require another Plan. Work kinds can recur at different scopes and participate in several contexts.

The property to preserve is not one universal order. It is that the applicable route, dependencies, bypasses, evidence, authority, and reopenings remain explicit and historically reconstructable.

18. Decomposition without loss of lineage

Complex work must be decomposed into bounded assignments. Each assignment should receive an immediate objective, selected context, constraints, an expected result, applicable gates, and an authority boundary.

The assignment should not need to carry the whole project in its prompt or working memory. It should retain a resolvable path to the larger purpose while receiving only the context relevant to its responsibility.

```
context available through lineage  
!= context materialized for one execution
```

This distinction allows the broader context to remain preserved without making every local worker interpret everything.

19. Selecting context for local work

Too little context produces work that may be locally correct but globally harmful. Too much context creates cost, distraction, conflicting instructions, and unclear authority.

The system therefore needs a way to select bounded context for a task while preserving the path back to excluded context. Relevance does not automatically confer authority, and ancestry does not automatically grant access.

Which information must be materialized, which may remain available through lineage, and how a selection can be checked are open questions under `OD-02`.

20. Logical work, physical attempts, and effects

The system should distinguish:

- a logical task or operation;
- the person or agent assigned to it;
- a particular physical attempt;
- the result returned;
- the effect produced; and
- the artifact that records or embodies that effect.

A retry creates another attempt, not another purpose. A completed attempt does not automatically mean that its result was accepted. A mediated observation that a file changed is evidence of a state change. Only an accepted causal binding can attribute that change to a particular Attempt or intended effect; neither fact alone proves authorization, correctness, or usefulness.

These distinctions connect the conceptual work graph to observable execution without allowing runtime activity to redefine the work retroactively.

Alternative framings considered

Framing	Why it is insufficient on its own
One global lifecycle	Cannot naturally represent recursive Research, local Plans, and reopenings.
Give every worker the whole context	Expensive, distracting, and likely to blur responsibility and authority.
Treat retries as new tasks	Loses the distinction between logical work and physical execution.
Treat a produced artifact as completion	Confuses activity with acceptance and objective realization.

21. From activity to evidence

The system should not equate activity with progress.

```
work was requested
work started
an attempt returned
an artifact was produced
the artifact passed a local check
the result was accepted
the larger objective advanced
```

These are distinct claims. A workflow profile may require some as prerequisites for others, but no universal sequence or implication is assumed. Each claim needs its own evidence and any movement between them needs an accepted composition rule. Tests may show that code satisfies a local contract. They do not necessarily show that the contract still serves the feature, that the feature serves the Plan, or that the objective still matters.

22. Evidence that local work serves an objective

The downward realization path needs more than a chain of containment.

A task may be part of a feature without advancing it. A feature may belong to a project without serving the current objective. The system needs a way to represent and inspect the basis for claims that local work contributes to larger outcomes.

The required evidence will differ by level and kind of work. Some relationships may be supported by tests, others by review, observation, formal proof, or accountable human judgment. What counts as sufficient realization evidence remains open under `OD-03`.

23. Missing and partial grounding

The system must represent incompleteness honestly:

- an objective without any proposed realization route;
- a Plan without governing authority;
- a specification without implementation;
- implementation without verification;
- a task without an upward purpose path;
- an objective without downward realization;
- a claim without sufficient evidence; or
- a decision based on incomplete information.

These are legitimate states when they remain visible. They become misleading when the system fills the gaps through inference or displays them as if they were complete.

Alternative framings considered

Framing	Why it is insufficient on its own
Passing tests means the objective advanced	Tests usually establish a bounded technical claim, not strategic realization.
Every relation needs the same evidence	Different work and relation types require different warrants.
Hide incomplete objects until resolved	Removes the exact gaps the system should help users see.
Agent summary as grounding	A summary is another attributable assertion, not a substitute for source evidence.

24. When represented context diverges from work

The represented context can become stale:

- a task continues after its objective changes;
- a feature moves but its research remains attached to the old project;
- implementation follows a superseded specification;
- a Dispatch no longer fits the decision that was supposed to authorize it;
- an artifact changes without revisiting claims that depend on it.

This contextual drift is the gap between the relationships the system represents and the work that is actually occurring. Preserving history is not enough; the system also needs ways to detect when the current representation no longer fits.

A useful drift diagnosis localizes the break. It identifies the changed source, the dependent claim, the assumption that no longer holds, and the affected purpose, authority, assignment, causation, or realization path. The path may then be understood as current, missing, conflicted, or superseded without silently repairing its lineage.

25. Hooks, validators, and observers

Hooks can react when context changes. Validators can check explicit invariants. Observers can record what occurred.

These mechanisms may detect orphan work, broken lineage, stale authority, incompatible changes, missing evidence, or effects outside an accepted boundary. They should not manufacture semantic relationships merely because two files changed together or two events occurred nearby.

The hook is not the context. The validator is not the invariant. The observer is not the event it reports. Each is a mechanism participating in the preservation and inspection of context. When one accepted dependency changes, these mechanisms should help explain which dependent path changed and why, while leaving semantic repair to an attributable decision.

26. Current views from historical facts

The same accepted history can support several current views:

- work by objective;
- work by Plan or feature;
- active, blocked, abandoned, and complete work;
- tasks by agent or artifact type;
- decisions by scope;
- work with missing authority or evidence;
- physical folder and repository views.

These views should remain projections. A convenient dashboard, generated folder tree, or cached graph should not acquire authority merely because it is easier to inspect than its sources. A projection may legitimately omit information for its task, but it must not manufacture identity, fact, authority, causation, or currentness unsupported by accepted sources. Proximity in a view does not establish causation, and placement in a folder does not authorize action.

Alternative framings considered

Framing	Why it is insufficient on its own
Periodic manual cleanup	Repairs visible drift but does not provide continuous or attributable detection.
Hooks own the context	Confuses a reaction mechanism with the relationships and history it observes.
One canonical dashboard	Different questions require different projections over the same identities.
Projection becomes source of truth	Allows a derived view to manufacture or overwrite authority.

27. Candidate responsibilities

The system can first be understood as a collaboration among responsibilities rather than as a fixed collection of services:

1. **Intent capture** preserves incomplete objectives and their origin.
2. **Identity and relationship management** preserves addressable things and their typed connections.
3. **Decision and authority management** records acceptance, rejection, delegation, and effect permission.
4. **Planning and decomposition** turns intentions into bounded work.
5. **Context selection** materializes the relevant subset for each execution.
6. **Execution** performs work through people, agents, and tools.
7. **Evidence and verification** evaluates results and claims.
8. **History and projection** preserves accepted change and derives current views.
9. **Drift detection** checks whether represented context still matches ongoing work.

These responsibilities may share one implementation or be deployed separately. Their semantic boundaries matter before their deployment topology.

28. Distinctions a future ontology must own

A candidate substrate appears to need several distinctions without this essay settling their definitions. A future ontology must determine the boundaries among:

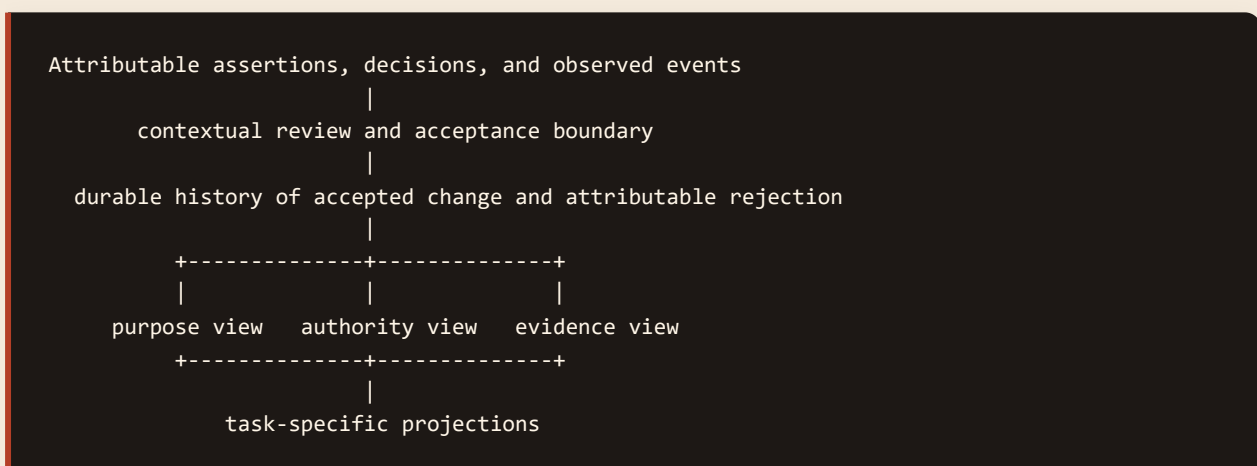
- addressable things that persist across descriptions and views;
- properties attributed to those things;
- connections carrying declared meaning;

- attributable claims about properties or connections;
- contextual acceptance, rejection, constraint, and authorization;
- accepted observations of change;
- durable content produced or consumed; and
- derived organizations of accepted facts.

These distinctions do not imply one database table or service per concept. This essay uses the provisional words object, description, relationship, assertion, decision, event, artifact, and projection only as navigation labels. Their meanings and boundaries remain deferred.

29. A graph family, a history, and several projections

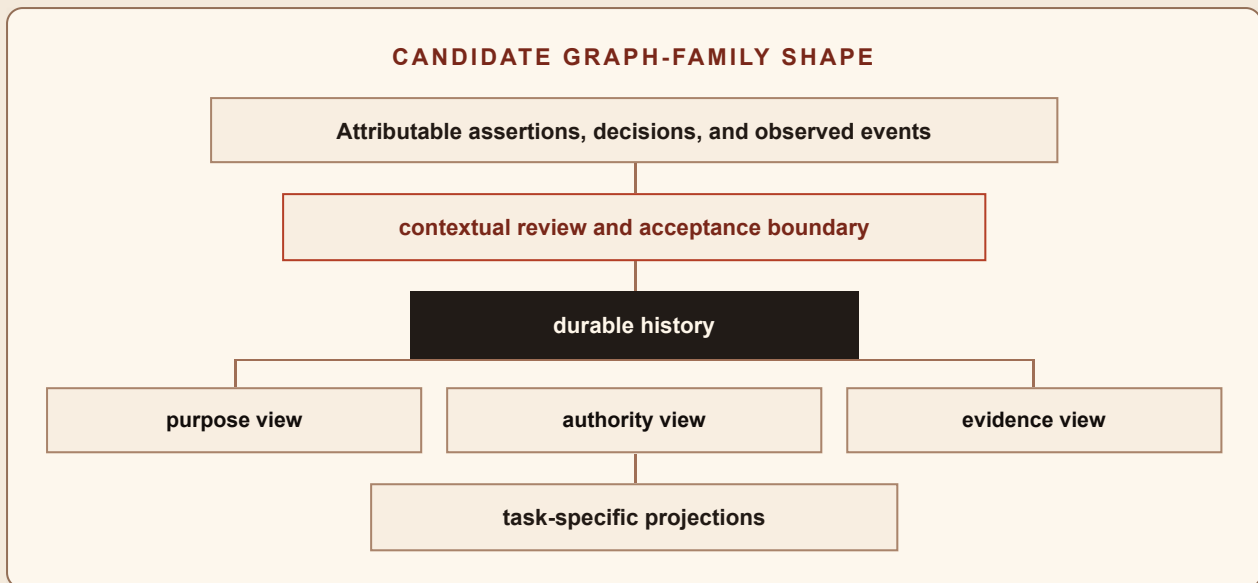
The emerging shape combines three different concerns:



This is a candidate graph-family shape, not a declaration that these three views form a mandatory taxonomy. Identity-bearing objects and attributable direct relations provide source material. Contextual review separates accepted change from rejected or unresolved claims without erasing the attributable occurrences. Overlapping graph views and task-specific projections can expose facts applicable under their declared acceptance stance without becoming their source. Authority constrains which changes and effects are allowed. Evidence supports or challenges assertions within the structure.

Whether history is persisted first, graph state is persisted first, or each is maintained through a witnessed hybrid remains open. The required property is reconstructability without allowing a current graph or projection to rewrite the history from which its claims are justified.

No one representation is automatically complete. A graph may preserve relationships while an artifact preserves content that should not be reduced to nodes and edges. The architecture should connect representations without pretending they are interchangeable.



30. Composition and derived meaning

Some relation paths may license derived conclusions:

```
Task implements Specification
Specification defines Feature
Feature serves Objective
```

Even here, the conclusion that the Task serves the Objective depends on accepted composition rules, applicable versions, and direct witnesses.

Other paths must not compose:

```
Task authorized-by Decision
Decision part-of Plan
```

This does not automatically mean that the Plan authorized the Task. Relation meaning, direction, scope, transitivity, and composition policy determine what can be derived.

Direct relations remain attributable historical source material. A derived conclusion additionally needs a replayable witness that can answer which direct sources were used, which composition stance made the path admissible, which scope and version applied, and why the conclusion held at that time. If a rule or acceptance changes, the derived conclusion may cease to apply without rewriting its direct sources. The design of these witnesses and rules remains open under **OD-04**.

Alternative framings considered

Framing	Why it is insufficient on its own
Permanent architecture layers	The responsibility boundaries are candidates, not yet deployment decisions.
Reduce every artifact to a graph	Graphs represent selected structure, not all content or behavior.
Make every relation transitive	Produces invalid purpose and authority conclusions.
One central service owns all meaning	Risks turning an interoperability substrate into a semantic and operational monolith.

31. Candidate invariants

Before choosing a kernel, the system can name properties it may need to preserve:

- names, paths, classifications, and projections do not silently redefine identity;
- accepted history remains reconstructable;
- relationships preserve type, direction, provenance, scope, and applicable version;
- derived paths retain their direct witnesses and composition rules;
- projections cannot manufacture facts or authority;
- lineage does not automatically delegate tools, evidence, budget, or effect permission;
- cheap descriptions do not silently become types, truths, or permissions;
- revocation and supersession remain visible;
- checker acceptance does not imply physical enforcement; and
- local validity does not automatically imply global compatibility.

These are candidate invariants. Naming them does not prove that they are necessary, sufficient, or enforced.

32. One trusted boundary, several, or protocols

One open question is whether bounded acceptance and checking responsibilities should be organized through something called a kernel at all. Candidate system shapes include:

- one universal kernel;
- several specialized kernels;
- a microkernel with extensions;
- a kernel-of-kernels governing compatibility;

- protocols connecting independently governed domains; or
- an architecture in which kernel is not a useful organizing concept.

The choice depends on which invariants are genuinely global, which remain domain-specific, and what must be witnessed when independently valid domains compose. This remains open decision

OD-05 .

33. Formalization

Formal methods may help with properties that can be stated precisely:

- identity preservation;
- relation signatures and permitted composition;
- acyclicity or controlled cycles;
- authority containment;
- historical consistency;
- projection preservation;
- compatibility among independently governed rule sets.

Different tools establish different warrants. Deterministic validators can check concrete data against explicit rules. SMT solvers can search bounded constraint spaces. Lean can express and check selected mathematical claims. Tests can establish behavior for exercised cases. Reviews and human decisions can address judgments that are not reduced to those mechanisms.

No one tool establishes that the overall product objective is worthwhile, that a formal model faithfully represents reality, or that an accepted rule is physically enforced in every runtime.

34. A gate for category-theoretic interpretation

The candidate substrate described so far is no more than a typed multigraph with selected relations and possible composition rules. It has not established the identities, closure, associativity, or coherence required to treat any relation family as a category.

Category theory may become useful for a bounded relation family only after that family names:

- its carriers and directed transformations;
- identity transformations;
- a closed and associative composition law;
- the structure a proposed projection must preserve; and

- a counterexample or collapse test that would reject the interpretation.

For example, the path from Task to Specification to Feature to Objective could become a candidate only after the relevant relation family states which arrows compose and what conclusion that composition warrants. Until then it remains a witnessed path in a typed graph.

More elaborate candidates—including functors between contexts, natural transformations, limits, colimits, and residue—remain outside this architecture vocabulary until a named obligation requires them and supplies the corresponding laws and falsifier. Mathematical elegance alone does not show that a running orchestrator obeys the model.

Alternative framings considered

Framing	Why it is insufficient on its own
Start with the kernel	Chooses a mechanism before establishing the property it must preserve.
Treat every candidate invariant as universal	Converts open design questions into hidden governing law.
Formal proof as total product evidence	A proof establishes its formal statement under assumptions, not empirical fit or runtime enforcement.
Category theory as metaphor only	Risks adding vocabulary without changing a design decision or enabling a check.

35. The smallest useful vertical slice

The first system should test one end-to-end question:

Given one candidate outcome, observed artifact, or observed effect, can the system first determine whether accepted terminal evidence exists and then reconstruct defensible purpose, authority, assignment, and causal execution paths without treating adjacency as proof?

The purpose and authority paths converge but do not collapse:

```
purpose:    objective -> chosen route -> local work
authority:   accepted permission -> authorized execution -> attempt
assignment:  local work -> bounded responsibility -> attempt
causation:   attempt -> observed contribution -> candidate outcome
realization: local work -> supported outcome claim -> candidate outcome
```

The first slice should test whether an explicit assignment bridge is required. Convergence on the same outcome supplies a counterexample candidate when an authorized attempt was carrying a different responsibility: the test should determine whether the execution basis must connect that attempt to the purpose-bearing local work rather than relying on a shared outcome alone. The necessity and sufficiency of that bridge remain open under `OD-06`.

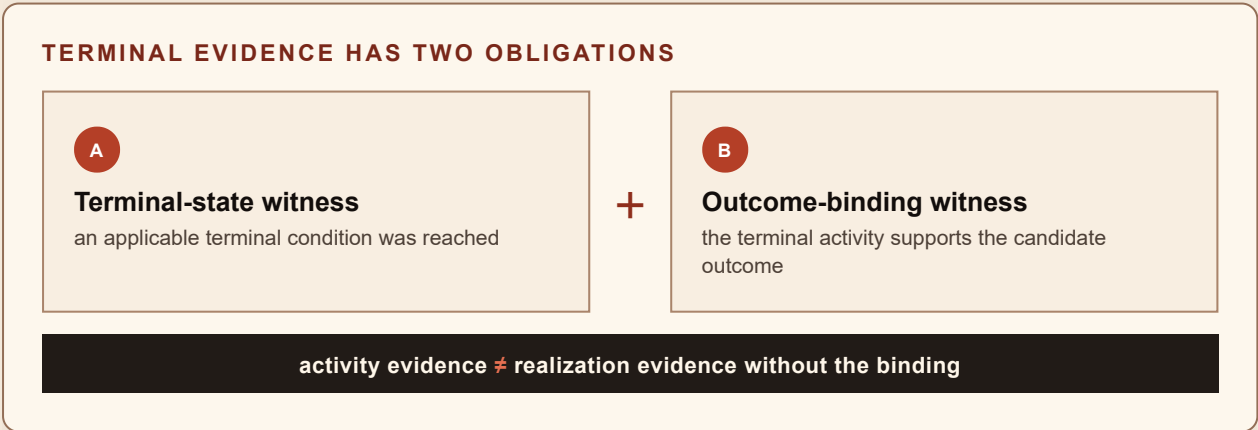
Terminal evidence separates two obligations. A **terminal-state witness** shows that an attempt or effect route reached an applicable terminal condition. An **outcome-binding witness** connects that terminal activity to the candidate outcome. The first without the second is activity evidence, not causal or realization evidence. Likewise, finalized content without an accepted producer binding is durable content rather than realization evidence, and an intended effect without an accepted outcome observation remains a request.

For any accepted causal binding, the slice should expose its attributable source, the acceptance occurrence or process, the scope and version in which it applies, and resolvable supporting evidence. Which process may accept such a binding and what evidence is sufficient remain open.

The slice should support traversal in both directions, expose missing links instead of filling them, and preserve historical changes to each path. It should not require a universal ontology, a final trusted-boundary design, or formalization of every relation before testing the core value.

Current repository correspondence for the first slice

The present repository specifies two non-canonical correspondence variants. For runtime-managed flows, ACI specifies ownership of confirmed Dispatch authority, and APT specifies the corresponding `aci_managed` authority-snapshot reference. For legacy-managed flows, ACI specifies that authority remains with the legacy ledger, while APT specifies a `legacy_ledger` external-owner reference rather than duplicated authority. These names describe current repository contracts; they do not select the general architecture, establish a future contract, or prove that a runtime exists.



36. What the first slice should test

The slice should begin with one fully witnessed golden case. Negative cases should be derived by changing one binding at a time: remove assignment, supersede purpose or authority, break accepted causation, omit terminal or outcome evidence, or introduce an ambiguous nearby relation that must not be inferred. One case should combine independently valid domain judgments that do not license their composition.

For each mutation, the test should traverse upward and downward, identify the first broken binding, explain why that path changed, and show that unrelated paths did not change. Expected answers and the evidence needed to reach them should be frozen before evaluation. The same questions should then be reconstructed from ordinary repository artifacts without the lineage representation.

The evaluation should measure answer classification and first-break localization; investigation time and evidence sources inspected; capture and repair burden; false confidence; and the comparison with ordinary repository reconstruction. Capture and repair cost may reverse an adoption decision even when answer quality improves. The slice fails if it:

- confuses purpose with authority;
- mistakes adjacency for causation;
- accepts an authorized Attempt as realizing a Task it was not assigned to execute;
- hides a missing or superseded link;
- cannot distinguish requested effects from completed outcomes; or
- cannot localize a one-binding mutation without disturbing unrelated paths;
- produces no measurable explanatory or decision benefit at an acceptable capture and repair cost.

Exact thresholds and workload accounting remain experiment decisions, but they must be frozen before the slice is evaluated.

37. Open questions

- Which things deserve stable identities, and at what point?
- Which relationships are direct facts, and which may be derived?
- What makes an upward path sufficient to justify local work?
- What evidence shows that micro work advanced a macro objective?
- When does changed context invalidate earlier authority?
- How much context should a leaf worker receive?

- Can an object participate in several Plans or projects without ambiguous authority?
- Which candidate invariants are global, domain-local, or configurable?
- What must be checked by validators, formal methods, empirical tests, or human judgment?
- How should independently governed contexts compose?
- How can maintenance cost and recovered value be measured?

38. What this view does not decide

This view names the system shape and its load-bearing questions. It does not decide their verdicts.

Open decision	Tension	Downstream owner state
OD-01 authority containment	useful bounded delegation vs accidental authority expansion	unassigned; future engineer-view work
OD-02 context selection	minimal local context vs sufficient purpose and constraint	unassigned; future engineer-view work
OD-03 objective-realization evidence	inspectable contribution vs unprovable strategic causality	unassigned; future engineer-view work
OD-04 relation composition	useful derived paths vs manufactured conclusions	unassigned; future engineer-view work
OD-05 trusted-boundary shape	shared trusted core vs independently governed domains	unassigned; future engineer-view work
OD-06 assignment binding	explicit responsibility continuity vs unnecessary mandatory linkage	unassigned; future engineer-view work

The table records missing ownership rather than pretending that downstream rows already exist. Record shapes, failure vocabularies, storage, runtime mechanics, and verdicts on these decisions remain outside this essay.

The following term meanings are also deferred rather than owned here:

Provisional navigation term	Definition owner state
identity, context, relationship, assertion	unassigned future ontology-view work
decision, authority, event	unassigned future ontology-view work
artifact, projection, invariant, kernel	unassigned future ontology-view work

Evidence boundary

Claim family used by this essay	Repository owner or evidence
Plan identity, nesting, and authority boundary	Plan contract
Root infrastructure objective and proposal-only authority	Governed Agent Work Infrastructure Plan
Proposal-only peer for broader language, context, relation, authority, and architecture shape	Agent-language system view
Macro-to-micro problem statement and changing-context questions	Linking the Macro to the Micro
Candidate primitives, graphs, services, and invariants	From Macro-to-Micro Context to Governed Primitives
Current Dispatch authority and Attempt/effect/artifact distinctions	ACI domain specification
Session, Dispatch, research, and authority-snapshot provenance	APT domain specification

The ordering, responsibility model, candidate invariants, open-decision inventory, and falsification-oriented first slice are new synthesis and remain proposal-only.

Alternative framings considered

Framing	Why it is insufficient on its own
Build the general ontology first	Delays testing whether the macro-to-micro path produces practical value.
Implement only the current Dispatch lineage	Proves execution ancestry but may leave purpose and decision ancestry disconnected.
Judge success by graph completeness	Rewards recording more structure rather than improving explanation, control, or decisions.
Treat this view as an accepted architecture	Converts a shape-and-stakes proposal into an unauthorized implementation route.

System-view result

- **Status:** flag — complete as a companion essay, but its open decisions and ontology terms do not yet have instantiated downstream owners.
- **Target boundary:** macro-to-micro work context from human objective through local effect, including purpose, authority, evidence, change, candidate architecture, and a first vertical slice.
- **Stakeholder altitude:** begins with a general reader and increases toward product and architecture stakeholders; schema and runtime implementation are deferred.
- **Shape:** perception -> intention -> decomposition -> context preservation -> authority and evidence -> architecture responsibilities -> invariants and formalization -> first slice.
- **Layering:** human situation and current work are given; context selection, decomposition, governance, and projections are optimized; accepted history accumulates.
- **Open decisions named:** six, all explicitly unassigned.
- **Decided-nothing check:** pass — the named stances remain open.
- **Term-deferral check:** pass with dependency — provisional navigation terms are explicitly deferred to unassigned future ontology work.
- **Evidence boundary:** repository-backed problem and current mechanisms; proposed synthesis for the full architecture and validation route.